

A SMALL THEME FOR MODERN SLIDES

Infrastructure & DevOps

Docker, VPS, Nginx/SSL, workers, and monitoring

Weblearns Academy

Odoo Full-Stack Developer Program

Senior Developer Lecture Materials

July 2026

WL

Agenda

- 1 Containers and Compose for Odoo
- 2 Hardened VPS and Nginx reverse proxy
- 3 systemd, backups, and TLS
- 4 Workers, indexes, and observability

TOC

How this deck is taught

PATTERN

Objectives → teaching → examples →
pitfalls → lab

LIVE CODING

Type commands with learners; freeze a
reference commit

SAFETY

Use disposable DBs; never demo sudo
shortcuts in production

Docker

Nginx

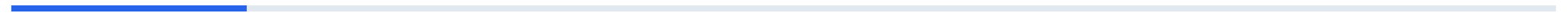
SSL

Workers



1 I01 – I05

Infrastructure & DevOps · teaching block 1



Containers vs Virtual Machines

- Explain the difference between OS virtualization and process isolation.
- Choose an appropriate runtime for Odoo development and deployment.
- Identify what containers do not solve.

Lab target — Draw an architecture for local Odoo using containers and explain which files survive when the Odoo container is deleted.

Containers vs Virtual Machines

Containers vs Virtual Machines is infrastructure work that directly affects reliability, data safety, and developer speed. Containers package a process with its filesystem and dependencies, while virtual machines package an entire guest operating system. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

Containers vs Virtual Machines (continued)

The design decision starts by separating local development, staging, and production needs. Use containers to make Odoo services repeatable and portable, but still design the host, network, storage, and backup strategy deliberately. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Containers vs Virtual Machines (continued)

Implementation should be explicit and repeatable. Run Odoo, PostgreSQL, Nginx, and workers as separate services where appropriate and keep persistent data outside disposable container layers. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Containers vs Virtual Machines (continued)

Verification must prove behavior, not only syntax. Verify isolation assumptions by restarting containers, removing images, and confirming that database and filestore data remain intact. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

Containers vs Virtual Machines (continued)

In production, containers vs virtual machines choices become long-lived operational contracts. Containers reduce drift, but they do not replace monitoring, patching, secrets management, or restore testing. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- Containers share the host kernel.
- VMs provide stronger OS-level separation.
- Container filesystems are disposable by design.
- Persistent data belongs in volumes or managed storage.
- Containers improve repeatability, not application design.

Mental model comparison

Mental model comparison

The model highlights why containers start quickly but still depend on host and storage decisions.

```
# Container mindset  
app process + image filesystem + mounted volumes + network namespace  
  
# VM mindset  
guest OS + kernel + virtual disk + virtual network + app processes
```

Common mistakes

- Storing database data inside a disposable container layer.
- Assuming containers are automatically secure.
- Using one giant container for every service.
- Forgetting host-level patching and monitoring.

Caution — Validate fixes in a disposable database before production.

Installing Docker Compose for Odoo Workloads

- Install Docker Engine and Compose plugin.
- Verify versions and permissions.
- Prepare a host for repeatable compose projects.

Lab target — Install Docker and Compose on a disposable VM, run hello-world, and document the exact version output.

Installing Docker Compose for Odoo Workloads

Installing Docker Compose for Odoo Workloads is infrastructure work that directly affects reliability, data safety, and developer speed. Docker Compose is the most common entry point for running Odoo, PostgreSQL, and supporting services together on a single host. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

Installing Docker Compose for Odoo Workloads (continued)

The design decision starts by separating local development, staging, and production needs. Decide whether the host is for development or production because package sources, user permissions, filesystem paths, and firewall rules differ. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Installing Docker Compose for Odoo Workloads (continued)

Implementation should be explicit and repeatable. Install Docker from official packages, enable the service, add only trusted operators to the docker group, and verify compose is available as docker compose. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Installing Docker Compose for Odoo Workloads (continued)

Verification must prove behavior, not only syntax. Run hello-world, inspect docker info, and create a minimal compose project to prove the daemon, plugin, network, and volume drivers work. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

Installing Docker Compose for Odoo Workloads (continued)

In production, installing docker compose for odoo workloads choices become long-lived operational contracts. A clean Docker installation prevents confusing deployment bugs where the application is blamed for host-level package or permission problems. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- Use the Compose plugin command: `docker compose`.
- Membership in the `docker` group is root-equivalent.
- Pin host setup in documentation or automation.
- Verify daemon and compose versions.
- Keep production hosts minimal.

Ubuntu installation outline

Ubuntu installation outline

The commands install Docker Engine and verify that the Compose plugin is available.

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
docker compose version
```

Common mistakes

- Installing old docker-compose v1 and mixing command syntax.
- Adding every user to the docker group casually.
- Skipping a reboot or new login after group changes.
- Debugging compose files before verifying the daemon works.

Caution — Validate fixes in a disposable database before production.

Docker CLI, Images, Containers, and Docker Hub

- Use core Docker CLI commands confidently.
- Explain images, containers, tags, and registries.
- Inspect running Odoo services.

Lab target — Run an Odoo container, inspect its environment and mounts, then explain the difference between stopping and removing it.

Docker CLI, Images, Containers, and Docker Hub

Docker CLI, Images, Containers, and Docker Hub is infrastructure work that directly affects reliability, data safety, and developer speed. The Docker CLI is the operational vocabulary for seeing what is running, what image it came from, and where logs are produced. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

Docker CLI, Images, Containers, and Docker Hub (continued)

The design decision starts by separating local development, staging, and production needs. Use tags deliberately: latest is convenient for experiments but too ambiguous for production upgrade control. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Docker CLI, Images, Containers, and Docker Hub (continued)

Implementation should be explicit and repeatable. Pull images, run containers, inspect metadata, read logs, execute shells, and remove unused resources with clear intent. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Docker CLI, Images, Containers, and Docker Hub (continued)

Verification must prove behavior, not only syntax. Verify the image digest, environment, mounts, and network settings when a container behaves differently from another host. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

Docker CLI, Images, Containers, and Docker Hub (continued)

In production, docker cli, images, containers, and docker hub choices become long-lived operational contracts. Production operators should be able to answer which image tag is running and how to recreate it before they make changes. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- Images are immutable templates.
- Containers are running or stopped instances.
- Registries store and distribute images.
- Tags are names, not guarantees of immutability.
- docker inspect is essential for debugging.

Useful Docker inspection commands

Useful Docker inspection commands

These commands show running services, images, recent logs, interactive access, mounts, and disk usage.

```
docker ps
docker images
docker logs --tail=100 odoo-web
docker exec -it odoo-web bash
docker inspect odoo-web --format '{{json .Mounts}}'
docker system df
```

Common mistakes

- Using latest tags in production.
- Removing volumes while trying to clean images.
- Ignoring image provenance.
- Debugging from assumptions instead of docker inspect output.

Caution — Validate fixes in a disposable database before production.

Dockerfile for Odoo

- Build a custom Odoo image.
- Install Python and system dependencies predictably.
- Keep addons and requirements layered for cache efficiency.

Lab target — Create a Dockerfile for custom addons, build it locally, and run Odoo from the image without mounting addon code.

Dockerfile for Odoo

Dockerfile for Odoo is infrastructure work that directly affects reliability, data safety, and developer speed. A custom Dockerfile turns an Odoo deployment from a mutable server into a versioned artifact. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

Dockerfile for Odoo (continued)

The design decision starts by separating local development, staging, and production needs. Decide which dependencies belong in the image and which configuration belongs at runtime through environment or mounted files. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Dockerfile for Odoo (continued)

Implementation should be explicit and repeatable. Start from a trusted Odoo base image, copy requirements before addons for cache efficiency, install packages as root, and run Odoo as the intended user. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Dockerfile for Odoo (continued)

Verification must prove behavior, not only syntax. Build the image from a clean context and run it without bind mounts to prove the image contains everything needed. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

Dockerfile for Odoo (continued)

In production, dockerfile for odoo choices become long-lived operational contracts. A production image should be reproducible, small enough to deploy quickly, and explicit about every dependency that custom code requires. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- Images should include code and dependencies, not secrets.
- Layer order affects build cache.
- Use requirements.txt for Python dependencies.
- Keep runtime config outside the image.
- Build and test images in CI.

Custom Odoo image

Custom Odoo image

The image installs Python dependencies first and then copies custom addons with the correct ownership.

```
FROM odoo:18.0

USER root
COPY requirements.txt /tmp/requirements.txt
RUN pip3 install --no-cache-dir -r /tmp/requirements.txt

USER odoo
COPY --chown=odoo:odoo ./addons /mnt/extra-addons
ENV ODoo_RC=/etc/odoo/odoo.conf
```

Common mistakes

- Copying secrets into the image.
- Installing dependencies interactively inside a running container.
- Invalidating the build cache by copying the whole repository too early.
- Running the final container as root without need.

Caution — Validate fixes in a disposable database before production.

Volumes for PostgreSQL, Filestore, and Addons

- Map persistent Odoo data to volumes.
- Separate database, filestore, and code storage.
- Avoid accidental data loss during container replacement.

Lab target — Attach a PDF to an Odoo record, recreate the Odoo container, and confirm the attachment still opens.

Volumes for PostgreSQL, Filestore, and Addons

Volumes for PostgreSQL, Filestore, and Addons is infrastructure work that directly affects reliability, data safety, and developer speed. Odoo persistence is split between PostgreSQL rows and the filestore, and both must be backed up together. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

Volumes for PostgreSQL, Filestore, and Addons (continued)

The design decision starts by separating local development, staging, and production needs. Design storage paths so database data, Odoo filestore, configuration, logs, and custom addons have clear ownership and backup policy. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Volumes for PostgreSQL, Filestore, and Addons (continued)

Implementation should be explicit and repeatable. Use named volumes or host bind mounts intentionally, and never rely on the writable container layer for anything important. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Volumes for PostgreSQL, Filestore, and Addons (continued)

Verification must prove behavior, not only syntax. Verify by deleting and recreating the application container while confirming that database records and attachments still exist. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

Volumes for PostgreSQL, Filestore, and Addons (continued)

In production, volumes for postgresql, filestore, and addons choices become long-lived operational contracts. Production restore depends on matching database dumps with the correct filestore, so volume layout is a backup design decision. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- PostgreSQL data and Odoo filestore are both persistent.
- Named volumes are managed by Docker.
- Bind mounts expose host paths explicitly.
- Container layers are disposable.
- Backups must include database and filestore together.

Compose volumes for Odoo data

Compose volumes for Odoo data

The database and filestore use persistent volumes while custom addons are mounted read-only.

```
services:
  db:
    image: postgres:16
    volumes:
      - pgdata:/var/lib/postgresql/data
  odoo:
    image: odoo:18.0
    volumes:
      - odoo-data:/var/lib/odoo
      - ./addons:/mnt/extra-addons:ro

volumes:
  pgdata:
  odoo-data:
```


Common mistakes

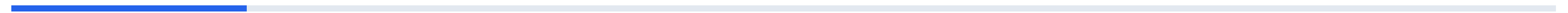
- Backing up only the database and forgetting attachments.
- Mounting writable addon code in production.
- Using anonymous volumes without knowing their names.
- Deleting volumes during image cleanup.

Caution — Validate fixes in a disposable database before production.



2 I06 – I10

Infrastructure & DevOps · teaching block 2



Docker Networking for Odoo

- Explain service discovery in Docker Compose.
- Connect Odoo to PostgreSQL by service name.
- Expose only required ports to the host.

Lab target — Create a compose network where only Odoo is published, then prove PostgreSQL is reachable from Odoo but not from the host port list.

Docker Networking for Odoo

Docker Networking for Odoo is infrastructure work that directly affects reliability, data safety, and developer speed. Compose networking gives services DNS names and private connectivity without exposing every port to the outside world. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

Docker Networking for Odoo (continued)

The design decision starts by separating local development, staging, and production needs. Decide which services need host access, which only need internal network access, and which should be isolated from public traffic. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Docker Networking for Odoo (continued)

Implementation should be explicit and repeatable. Use service names such as db as hostnames, publish only Nginx or Odoo ports needed by the environment, and avoid hard-coded container IP addresses. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Docker Networking for Odoo (continued)

Verification must prove behavior, not only syntax. Verify connectivity with `docker exec`, `getent hosts`, and application logs rather than guessing at network failures. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

Docker Networking for Odoo (continued)

In production, docker networking for odoo choices become long-lived operational contracts. A clean network design reduces attack surface and makes local, staging, and production compose files easier to compare. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- Compose services share a project network by default.
- Service names resolve through Docker DNS.
- ports publishes to the host; expose documents internal ports.
- Do not depend on container IP addresses.
- Limit public exposure.

Internal database network

Internal database network

Odoo connects to PostgreSQL using the db service name without publishing PostgreSQL to the host.

```
services:
  odoo:
    image: odoo:18.0
    environment:
      HOST: db
      USER: odoo
      PASSWORD: odoo
    ports:
      - "8069:8069"
    depends_on:
      - db
  db:
    image: postgres:16
    environment:
# ... truncated for slide
```

Common mistakes

- Publishing PostgreSQL publicly without a reason.
- Using container IPs that change after recreation.
- Assuming depends_on means the database is ready for connections.
- Mixing unrelated stacks on the same exposed ports.

Caution — Validate fixes in a disposable database before production.

Docker Compose Fundamentals

- Read and write compose service definitions.
- Use environment, volumes, ports, and depends_on.
- Operate a compose project safely.

Lab target — Write a minimal compose file with Odoo and PostgreSQL, run docker compose config, and explain every generated service field.

Docker Compose Fundamentals

Docker Compose Fundamentals is infrastructure work that directly affects reliability, data safety, and developer speed. Compose turns a multi-command deployment into a declarative project that can be started, stopped, inspected, and versioned. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

Docker Compose Fundamentals (continued)

The design decision starts by separating local development, staging, and production needs. Design compose files so the default command is safe for the target environment and overrides are explicit. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Docker Compose Fundamentals (continued)

Implementation should be explicit and repeatable. Define services, images or build contexts, environment variables, volumes, networks, healthchecks, and restart policies where appropriate. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Docker Compose Fundamentals (continued)

Verification must prove behavior, not only syntax. Verify with config rendering, logs, service status, and restart tests because valid YAML can still describe a poor deployment. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

Docker Compose Fundamentals (continued)

In production, docker compose fundamentals choices become long-lived operational contracts. Production compose usage should be predictable enough that an engineer can recreate the stack from the repository and documented secrets. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- docker compose config validates and expands configuration.
- Services are the main units.
- Environment can come from .env and env_file.
- Restart policy controls daemon behavior after failure.
- Compose project names affect resource names.

Common compose operations

Common compose operations

These commands validate, start, inspect, restart, and stop a compose project.

```
docker compose config
docker compose up -d
docker compose logs -f --tail=100 odoo
docker compose ps
docker compose restart odoo
docker compose down
```

Common mistakes

- Editing running containers instead of compose files.
- Skipping docker compose config before deployment.
- Confusing down with volume removal.
- Letting project names change between runs unexpectedly.

Caution — Validate fixes in a disposable database before production.

Multi-Container Odoo and PostgreSQL

- Run Odoo and PostgreSQL as separate services.
- Configure database credentials and health checks.
- Understand startup ordering and readiness.

Lab target — Run the two-container stack, restart PostgreSQL, and capture the relevant Odoo logs during reconnect.

Multi-Container Odoo and PostgreSQL

Multi-Container Odoo and PostgreSQL is infrastructure work that directly affects reliability, data safety, and developer speed. Separating Odoo and PostgreSQL mirrors the real dependency between an application server and a database server. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

Multi-Container Odoo and PostgreSQL (continued)

The design decision starts by separating local development, staging, and production needs. Choose explicit database credentials, persistent storage, healthchecks, and restart behavior before adding custom addons or proxy layers. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Multi-Container Odoo and PostgreSQL (continued)

Implementation should be explicit and repeatable. Use environment variables or config files for `db_host`, `db_user`, and `db_password`, and define a PostgreSQL healthcheck because `depends_on` is not readiness. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Multi-Container Odoo and PostgreSQL (continued)

Verification must prove behavior, not only syntax. Verify by restarting only the database, then only Odoo, and checking how each service logs reconnection or startup behavior. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

Multi-Container Odoo and PostgreSQL (continued)

In production, multi-container odoo and postgresql choices become long-lived operational contracts. A stable two-container baseline is the foundation for adding workers, Nginx, backups, and monitoring later. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- Odoo and PostgreSQL should be separate services.
- `depends_on` controls order, not readiness.
- Healthchecks make service state visible.
- Use persistent PostgreSQL storage.
- Restart behavior should be intentional.

Odoo and PostgreSQL compose stack

Odoo and PostgreSQL compose stack

The stack separates services and exposes database readiness through a healthcheck.

```
services:
  db:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: postgres
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 10s
      timeout: 5s
      retries: 5

# ... truncated for slide
```

Common mistakes

- Putting PostgreSQL inside the Odoo container.
- Using default passwords in shared environments.
- Assuming startup order means readiness.
- Restarting the whole stack for a single-service issue.

Caution — Validate fixes in a disposable database before production.

.env Files and Runtime Configuration

- Use .env files for non-secret and secret runtime values carefully.
- Understand Compose variable substitution.
- Separate committed examples from real credentials.

Lab target — Create .env.example and .gitignore entries for a compose project, then render the final compose config with local values.

.env Files and Runtime Configuration

.env Files and Runtime Configuration is infrastructure work that directly affects reliability, data safety, and developer speed. .env files make compose projects easier to configure, but they also become a common source of leaked secrets. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

.env Files and Runtime Configuration (continued)

The design decision starts by separating local development, staging, and production needs. Decide which variables are safe defaults, which are environment-specific, and which should come from a secret manager in production. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

.env Files and Runtime Configuration (continued)

Implementation should be explicit and repeatable. Commit `.env.example`, ignore real `.env` files, and use explicit variable names for database credentials, domain names, image tags, and worker settings. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

.env Files and Runtime Configuration (continued)

Verification must prove behavior, not only syntax. Verify the rendered configuration with docker compose config and check repository status before committing. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

.env Files and Runtime Configuration (continued)

In production, .env files and runtime configuration choices become long-lived operational contracts. A disciplined configuration approach lets staging and production differ without maintaining separate hand-edited compose files. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- .env supports Compose variable substitution.
- Commit examples, not real secrets.
- docker compose config shows resolved values.
- Use clear variable names.
- Production secrets may need a stronger mechanism.

.env.example

.env.example

The example documents required settings without pretending that the sample password is production-ready.

```
ODOO_IMAGE=odoo:18.0
POSTGRES_USER=odoo
POSTGRES_PASSWORD=replace-in-real-env
ODOO_DB_HOST=db
ODOO_WORKERS=4
ODOO_LIMIT_MEMORY_HARD=2684354560
DOMAIN=odoo.example.com
```

Common mistakes

- Committing real .env files.
- Assuming .env variables are automatically available inside containers.
- Using vague names such as PASSWORD for multiple services.
- Skipping rendered config review before deployment.

Caution — Validate fixes in a disposable database before production.

Upgrading Containers Safely

- Plan image and module upgrades.
- Back up database and filestore before changes.
- Roll forward or roll back from a known state.

Lab target — Write a one-page upgrade runbook for an Odoo container stack, including rollback criteria and validation checks.

Upgrading Containers Safely

Upgrading Containers Safely is infrastructure work that directly affects reliability, data safety, and developer speed. Container upgrades are easy to start and expensive to recover from if database migrations and filestore compatibility are ignored. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

Upgrading Containers Safely (continued)

The design decision starts by separating local development, staging, and production needs. Separate base image upgrades, custom addon changes, and database migrations into a planned sequence with a tested backup. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Upgrading Containers Safely (continued)

Implementation should be explicit and repeatable. Pull or build the new image, stop traffic, take backups, run Odoo update commands on staging first, and keep the previous image tag available. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Upgrading Containers Safely (continued)

Verification must prove behavior, not only syntax. Verify module updates, logs, scheduled jobs, reports, and core user journeys before reopening traffic. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

Upgrading Containers Safely (continued)

In production, upgrading containers safely choices become long-lived operational contracts. Safe upgrades are boring because every step has already been rehearsed on a restored copy of production. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- Pin image tags before upgrading.
- Back up database and filestore together.
- Test migrations on a restored copy.
- Keep previous image available for rollback.
- Review logs before declaring success.

Upgrade command sequence

Upgrade command sequence

The sequence backs up before starting the new image and runs an explicit module update.

```
docker compose pull odoo
docker compose stop odoo
./scripts/backup_odoo.sh
docker compose up -d odoo
docker compose exec odoo odoo -d prod -u training --stop-after-init
docker compose logs --tail=200 odoo
```

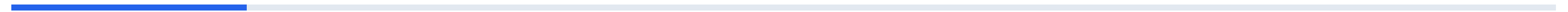
Common mistakes

- Using latest tags and not knowing what changed.
- Upgrading production before staging.
- Backing up the database without the filestore.
- Skipping post-upgrade functional checks.

Caution — Validate fixes in a disposable database before production.

3 I11 – I15

Infrastructure & DevOps · teaching block 3



CI/CD with GitHub Actions Basics

- Create a basic CI workflow for Odoo modules.
- Run linting and tests on pull requests.
- Build container images from commits.

Lab target — Create a CI workflow that compiles custom addon Python files and intentionally break one file to prove the check fails.

CI/CD with GitHub Actions Basics

CI/CD with GitHub Actions Basics is infrastructure work that directly affects reliability, data safety, and developer speed. CI/CD turns local confidence into repeatable checks that run before code reaches shared environments. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

CI/CD with GitHub Actions Basics (continued)

The design decision starts by separating local development, staging, and production needs. Start with fast checks that catch common failures: syntax, manifest validity, unit tests, and image build problems. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

CI/CD with GitHub Actions Basics (continued)

Implementation should be explicit and repeatable. Use GitHub Actions workflows with pinned action versions, service containers for PostgreSQL where needed, and artifacts for logs or coverage. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

CI/CD with GitHub Actions Basics (continued)

Verification must prove behavior, not only syntax. Verify that a failing test fails the workflow and that secrets are only available on trusted events. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

CI/CD with GitHub Actions Basics (continued)

In production, ci/cd with github actions basics choices become long-lived operational contracts. Production deployment should build on CI results, but the first goal is preventing broken custom addons from being merged unnoticed. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- Run checks on pull_request and push.
- Use PostgreSQL services for integration tests.
- Cache dependencies cautiously.
- Protect secrets from untrusted forks.
- Keep workflows readable and incremental.

Basic GitHub Actions workflow

Basic GitHub Actions workflow

The workflow starts with lightweight checks that catch syntax and manifest problems.

```
name: odoo-ci

on:
  pull_request:
  push:
    branches: [main]

jobs:
  python-checks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
# ... truncated for slide
```


Common mistakes

- Putting deployment secrets in pull_request workflows from forks.
- Running huge end-to-end jobs before cheap syntax checks.
- Ignoring workflow failure logs.
- Letting CI use a different dependency set than production.

Caution — Validate fixes in a disposable database before production.

VPS Provisioning and Production Directory Layout

- Prepare a VPS for Odoo deployment.
- Design a clear production directory structure.
- Separate app code, config, backups, logs, and secrets.

Lab target — Provision a small test VPS or VM and create the directory layout with ownership and permissions suitable for an odoo deploy user.

VPS Provisioning and Production Directory Layout

VPS Provisioning and Production Directory Layout is infrastructure work that directly affects reliability, data safety, and developer speed. A VPS is simple enough for small Odoo deployments but still needs disciplined layout and host preparation. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

VPS Provisioning and Production Directory Layout (continued)

The design decision starts by separating local development, staging, and production needs. Provision with a supported OS, enough memory and disk IOPS, a non-root deploy user, firewall rules, time sync, and monitoring access. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

VPS Provisioning and Production Directory Layout (continued)

Implementation should be explicit and repeatable. Create predictable directories such as `/opt/odoo/app`, `/opt/odoo/config`, `/opt/odoo/backups`, `/var/log/odoo`, and `/srv/odoo/filestore` depending on the deployment style. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

VPS Provisioning and Production Directory Layout (continued)

Verification must prove behavior, not only syntax. Verify permissions by running the service as the target user and confirming that backups can write only where intended. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

VPS Provisioning and Production Directory Layout (continued)

In production, vps provisioning and production directory layout choices become long-lived operational contracts. A good layout makes incidents calmer because engineers can find the compose file, config, backups, and logs without searching the whole server. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- Use a non-root deploy or odoo user.
- Keep configuration and secrets separate from code.
- Define backup and log paths explicitly.
- Document the directory layout.
- Provision disk and memory for growth.

Production layout example

Production layout example

The layout keeps operational assets separate and easy to back up or audit.

```
/opt/odoo/  
  app/          # compose file or deployment repository  
  config/       # odoo.conf, nginx snippets, env files  
  backups/      # database and filestore backups  
  scripts/      # backup, restore, health checks  
/var/log/odoo/  # application logs  
/srv/odoo/filestore/ # optional external filestore path
```

Common mistakes

- Running everything from a developer's home directory.
- Mixing backups into the application repository.
- Leaving important files owned by root after manual commands.
- Not documenting which path is authoritative.

Caution — Validate fixes in a disposable database before production.

SSH Hardening

- Reduce common SSH attack surface.
- Use key-based authentication.
- Protect administrative access without locking out operators.

Lab target — Harden SSH on a disposable VM, prove key login works in a second session, and document the rollback path.

SSH Hardening

SSH Hardening is infrastructure work that directly affects reliability, data safety, and developer speed. SSH is usually the front door to a VPS, so weak SSH posture undermines every application hardening effort. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

SSH Hardening (continued)

The design decision starts by separating local development, staging, and production needs. Design access around named users, public keys, sudo policy, and an emergency recovery path from the VPS provider console. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

SSH Hardening (continued)

Implementation should be explicit and repeatable. Disable password login, disable root login, restrict users where practical, install fail2ban or equivalent protection, and keep authorized keys reviewed. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

SSH Hardening (continued)

Verification must prove behavior, not only syntax. Verify a second session can connect before closing the first and test sudo access for the deploy process. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

SSH Hardening (continued)

In production, ssh hardening choices become long-lived operational contracts. Good SSH hardening balances security and operability: locked-out administrators cause downtime just as surely as attackers do. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- Use public key authentication.
- Disable direct root login.
- Keep an active session while testing changes.
- Limit sudo and user access.
- Maintain an emergency recovery path.

sshd hardening fragment

sshd hardening fragment

The fragment disables password and root login while allowing a named deploy user.

```
# /etc/ssh/sshd_config.d/10-hardening.conf
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AllowUsers deploy
ClientAliveInterval 300
ClientAliveCountMax 2
```

Common mistakes

- Restarting SSH after changes without an open backup session.
- Sharing one private key between operators.
- Disabling root login before creating a sudo-capable user.
- Forgetting provider console recovery.

Caution — Validate fixes in a disposable database before production.

Nginx Reverse Proxy for Odoo

- Proxy public HTTP traffic to Odoo.
- Set headers required for correct URL generation.
- Configure timeouts for long Odoo requests.

Lab target — Put Nginx in front of Odoo, enable `proxy_mode`, and confirm generated `web.base.url` uses the public hostname.

Nginx Reverse Proxy for Odoo

Nginx Reverse Proxy for Odoo is infrastructure work that directly affects reliability, data safety, and developer speed. Nginx usually terminates public HTTP traffic and forwards application requests to Odoo on an internal port. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

Nginx Reverse Proxy for Odoo (continued)

The design decision starts by separating local development, staging, and production needs. Decide where TLS terminates, which hostnames are valid, and whether Odoo runs in `proxy_mode` so it trusts forwarded headers correctly. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Nginx Reverse Proxy for Odoo (continued)

Implementation should be explicit and repeatable. Configure proxy_pass, Host, X-Forwarded headers, body size, buffering, and timeouts appropriate for Odoo imports and reports. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Nginx Reverse Proxy for Odoo (continued)

Verification must prove behavior, not only syntax. Verify real client IPs, generated URLs, file uploads, long report renders, and backend logs after enabling the proxy. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

Nginx Reverse Proxy for Odoo (continued)

In production, nginx reverse proxy for odoo choices become long-lived operational contracts. A correct reverse proxy prevents subtle bugs around redirects, mixed content, incorrect base URLs, and client IP audit logs. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- Enable proxy_mode in Odoo when behind a proxy.
- Forward Host and X-Forwarded headers.
- Set body size for imports and attachments.
- Tune timeouts for reports and upgrades.
- Keep upstream ports private when possible.

Nginx Odoo proxy server

Nginx Odoo proxy server

The proxy forwards standard headers and allows long Odoo requests to complete.

```
server {  
    listen 80;  
    server_name odoo.example.com;  
  
    client_max_body_size 100m;  
  
    location / {  
        proxy_pass http://127.0.0.1:8069;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
        proxy_read_timeout 720s;  
        proxy_connect_timeout 720s;  
    }  
    # ... truncated for slide
```

Common mistakes

- Forgetting `proxy_mode` in Odoo configuration.
- Leaving Odoo publicly exposed when Nginx should be the entry point.
- Using low timeouts that break large reports or imports.
- Not forwarding `X-Forwarded-Proto`, causing bad redirects.

Caution — Validate fixes in a disposable database before production.

Ports 80 and 443 to 8069, Longpolling 8072, and Let's Encrypt

- Route public HTTP and HTTPS traffic to Odoo.
- Handle longpolling or websocket-style notification endpoints.
- Install TLS certificates with Certbot.

Lab target — Configure HTTPS for a test hostname, add a longpolling location, and run `certbot renew --dry-run`.

Ports 80 and 443 to 8069, Longpolling 8072, and Let's Encrypt

Ports 80 and 443 to 8069, Longpolling 8072, and Let's Encrypt is infrastructure work that directly affects reliability, data safety, and developer speed. Users should reach Odoo on standard web ports while the application listens internally on ports such as 8069 and longpolling on 8072. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

Ports 80 and 443 to 8069, Longpolling 8072, and Let's Encrypt (continued)

The design decision starts by separating local development, staging, and production needs. Plan firewall rules, Nginx server blocks, TLS termination, renewal automation, and the route for bus or longpolling traffic. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Ports 80 and 443 to 8069, Longpolling 8072, and Let's Encrypt (continued)

Implementation should be explicit and repeatable. Use Certbot for Let's Encrypt certificates, redirect HTTP to HTTPS, and add a separate location for longpolling when the deployment uses a dedicated port. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Ports 80 and 443 to 8069, Longpolling 8072, and Let's Encrypt (continued)

Verification must prove behavior, not only syntax. Verify certificate renewal, live chat or bus notifications, redirect behavior, and firewall exposure from outside the server. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

Ports 80 and 443 to 8069, Longpolling 8072, and Let's Encrypt (continued)

In production, ports 80 and 443 to 8069, longpolling 8072, and let's encrypt choices become long-lived operational contracts. A deployment without reliable TLS and notification routing will look healthy until users report broken chat, Discuss, or real-time updates. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- Public users should use 80 and 443.
- Odoo backend often listens on 8069 internally.
- Longpolling commonly uses 8072 in worker deployments.
- Certbot can automate Let's Encrypt certificates.
- Test certificate renewal before relying on it.

Nginx HTTPS and longpolling locations

Nginx HTTPS and longpolling locations

The server routes normal requests to 8069 and longpolling requests to 8072 under HTTPS.

```
server {  
    listen 443 ssl http2;  
    server_name odoo.example.com;  
  
    ssl_certificate /etc/letsencrypt/live/odoo.example.com/fullchain.pem;  
    ssl_certificate_key /etc/letsencrypt/live/odoo.example.com/privkey.pem;  
  
    location /longpolling/ {  
        proxy_pass http://127.0.0.1:8072;  
        proxy_set_header Host $host;  
        proxy_set_header X-Forwarded-Proto https;  
    }  
  
    location / {  
# ... truncated for slide
```

Common mistakes

- Opening 8069 and 8072 publicly when only Nginx should be exposed.
- Forgetting automatic certificate renewal.
- Routing longpolling to the wrong upstream.
- Not testing real-time notifications after enabling workers.

Caution — Validate fixes in a disposable database before production.

4 I16 – I20

Infrastructure & DevOps · teaching block 4

systemd odoo.service

- Run Odoo as a systemd service.
- Define restart behavior and service user.
- Use journalctl for logs and diagnosis.

Lab target — Create a systemd unit on a test VM, start it, force a failure, and confirm restart behavior in journalctl.

systemd odoo.service

systemd odoo.service is infrastructure work that directly affects reliability, data safety, and developer speed. Some deployments run Odoo directly on a host instead of containers, and systemd is the standard process supervisor for that model. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

systemd odoo.service (continued)

The design decision starts by separating local development, staging, and production needs. Decide the service user, config file path, Python environment, addons path, restart policy, and log destination before writing the unit. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

systemd odoo.service (continued)

Implementation should be explicit and repeatable. Create a unit file with ExecStart, User, WorkingDirectory, Restart, and dependency on PostgreSQL or network availability as appropriate. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

systemd odoo.service (continued)

Verification must prove behavior, not only syntax. Verify daemon reload, enable, start, restart, failure restart, and journal logs so operations are repeatable. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

systemd odoo.service (continued)

In production, systemd odoo.service choices become long-lived operational contracts. A clear systemd unit makes a non-container deployment manageable and avoids fragile shell sessions or manual startup commands. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- Run Odoo under a dedicated user.
- Keep ExecStart explicit.
- Use systemctl enable for boot startup.
- Use journalctl -u for logs.
- Restart policy should match operational expectations.

Odoo systemd unit

Odoo systemd unit

The unit runs Odoo from a virtual environment with a dedicated user and restarts after failures.

```
[Unit]
Description=Odoo Application Server
After=network.target postgresql.service

[Service]
Type=simple
User=odoo
Group=odoo
WorkingDirectory=/opt/odoo/server
ExecStart=/opt/odoo/venv/bin/python3 /opt/odoo/server/odoo-bin -c /etc/odoo/odoo.conf
Restart=on-failure
RestartSec=5

[Install]
# ... truncated for slide
```

Common mistakes

- Running Odoo manually in screen or tmux as the production supervisor.
- Starting the service as root without need.
- Forgetting systemctl daemon-reload after editing the unit.
- Hiding logs in an unmanaged file path.

Caution — Validate fixes in a disposable database before production.

Backup Cron for Database and Filestore

- Back up PostgreSQL and Odoo filestore together.
- Schedule backups with cron.
- Test restore, retention, and failure alerts.

Lab target — Create a backup script and cron entry, then restore the output to a separate test database and open an attachment.

Backup Cron for Database and Filestore

Backup Cron for Database and Filestore is infrastructure work that directly affects reliability, data safety, and developer speed. Backups are only useful when they can be restored, and Odoo requires both database and filestore for a complete restore. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

Backup Cron for Database and Filestore (continued)

The design decision starts by separating local development, staging, and production needs. Design backup frequency, retention, encryption, offsite storage, and restore objectives before writing the cron line. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Backup Cron for Database and Filestore (continued)

Implementation should be explicit and repeatable. Use `pg_dump` or `pg_dumpall` as appropriate, archive the filestore matching the database, name backups with timestamps, and prune old files safely. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Backup Cron for Database and Filestore (continued)

Verification must prove behavior, not only syntax. Verify by restoring to a separate database and opening attachments, not by checking that a backup file exists. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

Backup Cron for Database and Filestore (continued)

In production, backup cron for database and filestore choices become long-lived operational contracts. Production backup policy should include alerting because a silent backup failure is discovered at the worst possible time. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- Database and filestore must be backed up as a pair.
- Timestamp backup names.
- Store copies off the application host.
- Test restore regularly.
- Alert on backup failures.

Backup script fragment

Backup script fragment

The script writes a timestamped database dump and matching filestore archive, then prunes old local backups.

```
#!/usr/bin/env bash
set -euo pipefail

DB=prod
STAMP=$(date +%Y%m%d-%H%M%S)
DEST=/opt/odoo/backups/$STAMP
mkdir -p "$DEST"

pg_dump -Fc "$DB" > "$DEST/$DB.dump"
tar -C /var/lib/odoo/filestore -czf "$DEST/filestore.tgz" "$DB"
find /opt/odoo/backups -mindepth 1 -maxdepth 1 -type d -mtime +14 -exec rm -rf {} +
```

Common mistakes

- Backing up only PostgreSQL.
- Never testing restore.
- Keeping all backups on the same disk as production.
- Writing cron jobs that fail silently.

Caution — Validate fixes in a disposable database before production.

Odoo Workers Architecture and Worker Calculation

- Explain threaded versus multi-worker Odoo modes.
- Calculate an initial worker count.
- Reserve capacity for cron and longpolling.

Lab target — Calculate initial workers for a 4-core, 8 GB VPS running Odoo and PostgreSQL, then justify memory reserves.

Odoo Workers Architecture and Worker Calculation

Odoo Workers Architecture and Worker Calculation is infrastructure work that directly affects reliability, data safety, and developer speed. Odoo workers let the application handle concurrent HTTP requests with multiple processes instead of one threaded development-style server. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

Odoo Workers Architecture and Worker Calculation (continued)

The design decision starts by separating local development, staging, and production needs. Base the initial worker count on CPU cores, memory, traffic profile, cron load, and whether the host also runs PostgreSQL. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Odoo Workers Architecture and Worker Calculation (continued)

Implementation should be explicit and repeatable. Set `workers`, `max_cron_threads`, `proxy_mode`, and `gevent` or `longpolling` configuration according to the Odoo version and deployment model. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Odoo Workers Architecture and Worker Calculation (continued)

Verification must prove behavior, not only syntax. Verify under load with realistic requests, not only by checking that processes exist, because workers can be CPU-bound, memory-bound, or database-bound. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

Odoo Workers Architecture and Worker Calculation (continued)

In production, odoo workers architecture and worker calculation choices become long-lived operational contracts. Worker sizing is an iterative capacity practice: start conservatively, measure, and adjust before users feel saturation. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- workers > 0 enables multi-process mode.
- A common starting formula is CPU cores * 2 + 1, adjusted for memory.
- Cron workers consume capacity.
- Longpolling or bus traffic may need separate handling.
- Measure before and after changes.

odoo.conf worker settings

odoo.conf worker settings

The settings enable worker mode with conservative time and memory limits for a modest host.

```
[options]
workers = 5
max_cron_threads = 1
proxy_mode = True
limit_time_cpu = 120
limit_time_real = 240
limit_memory_soft = 2147483648
limit_memory_hard = 2684354560
```


Common mistakes

- Using the CPU formula without checking memory.
- Forgetting cron capacity.
- Running many workers against too few PostgreSQL connections.
- Changing workers without load testing.

Caution — Validate fixes in a disposable database before production.

Memory Limits, Time Limits, PostgreSQL Pooling, and max_connections

- Configure Odoo request memory and time guards.
- Relate Odoo workers to PostgreSQL connections.
- Plan pooling or max_connections changes safely.

Lab target — Create a connection and memory budget for an 8-worker Odoo deployment and propose safe PostgreSQL max_connections.

Memory Limits, Time Limits, PostgreSQL Pooling, and max_connections

Memory Limits, Time Limits, PostgreSQL Pooling, and max_connections is infrastructure work that directly affects reliability, data safety, and developer speed. Odoo and PostgreSQL capacity problems often appear together because more workers can mean more database connections and more memory pressure. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

Memory Limits, Time Limits, PostgreSQL Pooling, and max_connections (continued)

The design decision starts by separating local development, staging, and production needs. Design limits from the host resources, request profile, scheduled jobs, and PostgreSQL memory settings instead of copying values from another deployment. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Memory Limits, Time Limits, PostgreSQL Pooling, and max_connections (continued)

Implementation should be explicit and repeatable. Set `limit_memory_soft`, `limit_memory_hard`, `limit_time_cpu`, `limit_time_real`, `db_maxconn`, and PostgreSQL `max_connections` with enough headroom for maintenance sessions. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Memory Limits, Time Limits, PostgreSQL Pooling, and max_connections (continued)

Verification must prove behavior, not only syntax. Verify by monitoring worker recycling, slow requests, database connection counts, and out-of-memory events under realistic load. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

Memory Limits, Time Limits, PostgreSQL Pooling, and max_connections (continued)

In production, memory limits, time limits, postgresql pooling, and max_connections choices become long-lived operational contracts. Production tuning is a balancing act: too loose risks host instability, too tight kills legitimate reports and imports. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- Odoo workers consume memory independently.
- db_maxconn controls Odoo's database connection pool per process.
- PostgreSQL max_connections must cover all clients.
- Memory limits recycle runaway workers.
- Long reports may need time limit review.

Connection budget example

Connection budget example

The calculation reminds operators to budget for all database clients, not only HTTP workers.

```
# Example budget
workers = 5
max_cron_threads = 1
db_maxconn = 16

# Potential Odoo connections: (workers + cron + main process) * db_maxconn
# Add admin tools, monitoring, backup jobs, and migration sessions before
# setting PostgreSQL max_connections.
```

Common mistakes

- Increasing workers without increasing database capacity.
- Setting max_connections high without enough PostgreSQL memory.
- Disabling time limits to hide slow code.
- Ignoring cron and migration connections.

Caution — Validate fixes in a disposable database before production.

Indexes, pg_stat_statements, Redis Sessions, Load Balancing, and Prometheus Grafana

- Identify slow SQL with pg_stat_statements.
- Explain when missing indexes hurt Odoo.
- Describe Redis sessions, load balancing, and monitoring basics.

Lab target — Enable pg_stat_statements on a staging database, identify one slow query pattern, and propose a measured index or code change.

Indexes, pg_stat_statements, Redis Sessions, Load Balancing, and Prometheus Grafana

Indexes, pg_stat_statements, Redis Sessions, Load Balancing, and Prometheus Grafana is infrastructure work that directly affects reliability, data safety, and developer speed. At scale, Odoo performance is usually constrained by database queries, worker saturation, cache behavior, or lack of visibility. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

Indexes, pg_stat_statements, Redis Sessions, Load Balancing, and Prometheus Grafana (continued)

The design decision starts by separating local development, staging, and production needs. Design observability before firefighting: know which metrics, logs, and query statistics reveal the difference between slow Python, slow SQL, and overloaded infrastructure. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Indexes, pg_stat_statements, Redis Sessions, Load Balancing, and Prometheus Grafana (continued)

Implementation should be explicit and repeatable. Enable pg_stat_statements, inspect high-total-time queries, add indexes only after understanding domains, and consider Redis-backed sessions or sticky load balancing for multi-node deployments. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Indexes, pg_stat_statements, Redis Sessions, Load Balancing, and Prometheus Grafana (continued)

Verification must prove behavior, not only syntax. Verify every performance change with before-and-after measurements, because indexes can slow writes and load balancers can break session assumptions. Test from a clean host or container, restart services, simulate failure paths, and confirm that logs and metrics expose enough information to diagnose issues.

Indexes, pg_stat_statements, Redis Sessions, Load Balancing, and Prometheus Grafana (continued)

In production, indexes, pg_stat_statements, redis sessions, load balancing, and prometheus grafana choices become long-lived operational contracts. Prometheus and Grafana provide the operational feedback loop: collect metrics, create dashboards, alert on symptoms, and use traces or logs for root cause. Document assumptions, keep rollback paths realistic, and automate the checks that humans are most likely to forget during a late maintenance window.

What to remember

- `pg_stat_statements` shows expensive query patterns.
- Indexes should match frequent domains and joins.
- Redis sessions can help multi-node session consistency.
- Load balancing requires shared filestore and session strategy.
- Prometheus and Grafana turn metrics into alerts and dashboards.

Inspect slow queries

Inspect slow queries

The query identifies expensive SQL patterns; the example index matches a common state and date domain.

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

SELECT calls,
       total_exec_time,
       mean_exec_time,
       rows,
       query
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;

CREATE INDEX IF NOT EXISTS training_course_state_start_idx
ON training_course (state, start_date);
```

Common mistakes

- Adding indexes blindly for every slow query.
- Load balancing without shared filestore or session planning.
- Creating dashboards with no alert thresholds.
- Ignoring write overhead from extra indexes.

Caution — Validate fixes in a disposable database before production.

Observability layers

Layer	Signal
Odoo	Request latency, worker count, errors
PostgreSQL	Slow queries, locks, connections
Host	CPU, memory, disk IO
Proxy	Status codes, upstream time

You should now be able to...

- I01 Containers vs Virtual Machines
- I02 Installing Docker Compose for Odoo Workloads
- I03 Docker CLI, Images, Containers, and Docker Hub
- I04 Dockerfile for Odoo
- I05 Volumes for PostgreSQL, Filestore, and Addons
- I06 Docker Networking for Odoo
- ... and 14 more lessons in this deck

Remember — Complete outstanding labs before starting the next section.

Docker

Nginx

SSL

Workers

NEXT STEP

Complete the section labs

Open the next presentation deck

Section 07 · Infrastructure

WL